

Code-Sage AI Software Architecture Document For the AI-Powered Technical-Debt Analytics Dashboard

Version 1.0

Group 16 | Project ID 7 | CS3203 - Software Engineering Project

Mentor: Mr. Anju Chamantha

Date: 9th August 2026

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Revision History

Date	Version	Description	Author
09/Aug/2026	1.0	This document establishes the initial v1.0 architecture (4+1 views, data model, and quality attributes). Aligned with SRS v1.0.	Group 16

Individual Contributions

Index No.	Name	Contributions
230432G	Nethmini R.M.N.	Prepared and documented Introduction, Use-Case View, Implementation View, Deployment View, Reliability, Scalability, AI Model Quality.
230435T	Nethsara U.D.K.C.	Prepared and documented Architectural Representation, Logical View, Deployment View, Data View, Performance, Security, Privacy, Portability.
230451M	Pabasara H.H.J.	Prepared and documented Architectural Goals and Constraints, Process View, Size and Performance, Usability, Supportability, Maintainability.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Table of Contents

1.	Introduction	5
1.1	Purpose	5
1.2	Scope	5
1.3	Definitions, Acronyms, and Abbreviations	5
1.4	References	6
1.5	Overview	6
2.	Architectural Representation	6
3.	Architectural Goals and Constraints	7
4.	Use-Case View	8
4.1	Sign In	9
4.2	Connect Repository	10
4.3	Run Repository Scan	10
4.4	View Health Dashboard	11
4.5	Triage & Filter Findings	12
4.6	Select / Configure Scoring Profile	12
4.7	Browse Scan History	13
5.	Logical View	14
5.1	Overview	14
5.2	Architecturally Significant Design Packages	15
6.	Process View	20
6.1	The extraction boundary	22
6.2	Applying a scoring profile	25
6.3	The visibility floor	27
7.	Deployment View	27
8.	Implementation View	29
8.1	Overview	29
8.2	Layers	30
8.2.1	Presentation Layer	31
8.2.2	Application Layer	31
8.2.3	Data Layer	31
8.2.4	External Systems	31
9.	Data View	32
10.	Size and Performance	35
11.	Performance Measurements	37
12.	Quality	37

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

12.1	Reliability	37
12.2	Performance	38
12.3	Security	38
12.4	Privacy	38
12.5	Usability	39
12.6	Maintainability	39
12.7	Supportability	39
12.8	Portability	40
12.9	Scalability	40
12.10	AI Model Quality	40

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

1. Introduction

1.1 Purpose

This document describes the software architecture of Code Sage AI, the AI-powered technical-debt analytics dashboard specified in the Software Requirements Specification. It uses the RUP "4+1" model, so the system is presented through five views - Use-Case, Logical, Process, Deployment and Implementation - together with a Data View, because the persistent schema in this system carries real architectural weight and leaving it out would hide a central decision.

The document has three audiences. The development team uses it to know which component owns which responsibility before writing code. The mentor and the evaluators use it to judge whether the design actually delivers what the SRS promises. Future maintainers use it to understand why the system is shaped this way, so that a later change does not quietly break an assumption the design depends on.

It should be read together with the SRS, which states what the system must do. This document states how the parts are arranged to do it. Where the two overlap, the SRS is the normative source, and this document refers back to the requirement by its identifier.

1.2 Scope

This document covers the architecture of the v1.0 release: a multi-tenant web application in which a user signs in with GitHub, connects a public repository by pasting its URL, runs a scan on a chosen branch when they decide to, and reads a prioritised technical-debt dashboard built from the stored result. It covers all four parts of the system - the frontend, the backend API and its asynchronous workers, the machine-learning service, and the database.

Three limits are stated here because they shape several of the views that follow.

- **Repository access in v1.0 is public repositories by URL only.** Sign-in uses GitHub OAuth, and the analysis pipeline reads the repository through an ordinary git clone. There is no GitHub App installation, no private-repository support and no webhook endpoint in v1.0. Those belong to later releases, and this document marks the points where they would attach (SRS FR-3, Table 3-21).
- **Scans are started by the user, never by an event.** There is no automatic or event-driven analysis in v1.0. A scan happens because a user asked for it (SRS FR-6).
- **Later-release concerns are out of scope.** Team management, role-based access control, private repositories and additional Git hosts are mentioned only where they affect a v1.0 decision - for example the tenant column that exists from day one so that multi-user workspaces can be added later without a migration (SRS FR-2, FR-23).

1.3 Definitions, Acronyms, and Abbreviations

The project keeps a single glossary in SRS Section 1.3, and that glossary is the authority. The terms used most often in this document are repeated below for the reader's convenience.

Table 1-1. Terms used most often in this document.

Term	Meaning in this document
Finding	One detected issue at a file:line:symbol, carrying a source, a category, a severity and a one-line reason. The atomic unit of output.
source	Which detector produced a finding. Exactly two values: rule or satd (SRS FR-8.2).
category	What kind of debt a finding is. Exactly five values: code-design, requirement, defect, documentation, test and security (SRS FR-9.3).
Snapshot	The stored, immutable result of one scan, identified by repository, branch, commit SHA and time.
Scoring profile	The five category weights plus the trust slider that turn stored findings into scores (SRS FR-20).

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Term	Meaning in this document
Visibility floor	The rule that a critical security finding stays visible no matter how the profile is set (SRS FR-24).
Workspace / tenant	The top-level owner of data. A project belongs to a workspace, not to a person.
RLS	PostgreSQL Row-Level Security, the mechanism used to keep tenants apart.
ML-1 / ML-2	The SATD comment classifier and the file-level bug-proneness risk model.

1.4 References

The documents referenced elsewhere in this SAD are listed below.

Table 1-2. Referenced documents.

#	Document	Where it is held
1	Software Requirements Specification, Code Sage AI v1.0. The normative source for every requirement identifier used in this document.	docs/Deliverables/Software_Requirements_Specification.docx
2	Change Request CR-001 - scoring model and finding UX (decisions D-CR1 to D-CR12).	docs/Change Requests/
3	Backend Analysis Engine design note.	docs/Project Management & Planning/
4	Release Roadmap and Frontend Prototype Plan.	docs/Project Management & Planning/
5	Project Proposal and Feasibility Report.	docs/Deliverables/
6	Shared data contract - the single source of truth for the shapes crossing the frontend, backend and database boundary (SRS SP-4).	apps/web/src/lib/types/index.ts

1.5 Overview

The rest of the document is organized as follows.

Section 2 explains which architectural views are used and why. Section 3 lists the goals and constraints that actually shaped the design, each with its architectural consequence. Sections 4 to 8 present the five views: the use cases that exercise the architecture, the decomposition into subsystems and classes, the runtime behaviour of the processes, the physical deployment, and the layering of the codebase. Section 9 describes the persistent data model and the rule that separates stored facts from derived scores. Section 10 covers sizing and performance targets, Section 11 explains how the architecture delivers the quality attributes, and Section 12 lists the references.

Ten figures are referenced by number throughout. Each one is cited in the text as "Figure n" and is described in the sentences that follow it.

2. Architectural Representation

CodeSage AI is a web-based software-as-a-service application designed for small scale agile software development teams to detect and prioritize technical debt in their code repositories hosted on GitHub. It uses a multi-tenant data architecture and each project, scan and finding belongs to a specific workspace. The initial version of the system limits each workspace to a single user. The system consists of a browser-based frontend, a backend API, asynchronous repository analyzing workers, offline trained machine-learning components and a database. The repository scans are initiated manually by users and executed in the background so that the user interface of the system remains responsive during the repository scan and analysis. The findings of the analysis are presented through an interactive dashboard

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

that displays repository health information, historical analysis results and a prioritized Refactor-First list that shows the findings that should be addressed first.

This document uses multiple views to represent the architecture of the application to describe the structural, behavioral, runtime, deployment and implementation aspects of the system. It follows the 4+1 architectural view approach that consists of Use-Case View, Logical View, Process View, Deployment View and Implementation View. A sperate Data View is included to represent the architectural decisions of the database that is used to store the persistent data of the application.

View	What it captures	Diagrams
Use-Case View	Describes the actors, external systems, use cases and relationships that represent architecturally significant interactions with the system	Use-case diagram, scenario tables (Figure 1)
Logical View	Describes the static structure of the system, including its main subsystems, packages, domain classes, responsibilities, attributes and relationships	class diagram (Figures 3)
Process View	Describes the runtime behavior of the system. This includes the API process, asynchronous workers, task queue, machine-learning inference, database communication and the sequence of activities of repository scanning and profile application	Activity and sequence diagrams (Figures 4, 5, 6)
Deployment View	Describes the physical and container-level deployment of the system. This includes client services, application containers, worker containers, the Redis broker and the network connections	Deployment diagram (Figure 7)
Implementation View	Describes how the software is organized in the source code including applications, architectural layers, packages, modules, components, boundaries between the presentation, application and data layers.	Component and package diagrams (Figures 8, 9)
Data View	Describes the persistent data model, including entities, attributes, relationships, tenant ownership, immutable scan snapshots, and the distinction between stored analysis facts and scores calculated when results are retrieved.	ER diagram (Figure 10)

3. Architectural Goals and Constraints

Table 3-1. Architectural goals and their consequences.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

#	Goal or constraint	What it forces in the architecture
G1	Secure multi-tenant architecture.	A workspace is a tenant. Workspace is protected by PostgreSQL Row-Level Security
G2	The UI must stay responsive during the scan.	Scans run on Celery workers behind a Redis broker, never inside the HTTP request. The API returns a scan ID immediately and the client polls for progress.
G3	Dashboard output should be low noise and explainable	Rule based thresholds and one line reason templates; Prioritized list for the findings
G4	Profile change does not require a complete re-scan	Scoring is done by backend engine at read time over stored findings. It does not perform a database write for each final dashboard outputs
G5	Severity never altered by users	Severity is assigned at development time and managed via config files. Users can only change scoring profiles
G6	Core features Should be developed first; Secondary features added top of existing core	Complete vertical slice of whole project scope will be developed first; Rule-based engine can work on its own. ML Services are delivered as separate services which can change without affecting core backend logic
G7	Free, containerized, independently scalable stack	Every component is delivered as Docker container. Workers scale horizontally to handle concurrent load.

Constraints. The technology stack is frozen: Next.js and Tailwind for the frontend; FastAPI, Celery and Redis for the backend; Python with scikit-learn, Tree-sitter, CK and PyDriller for metric extraction, analysis and ML; PostgreSQL for storage; Docker for deployment. Only supervised learning is used. Models are trained offline on public datasets and loaded at runtime as versioned artifacts.

4. Use-Case View

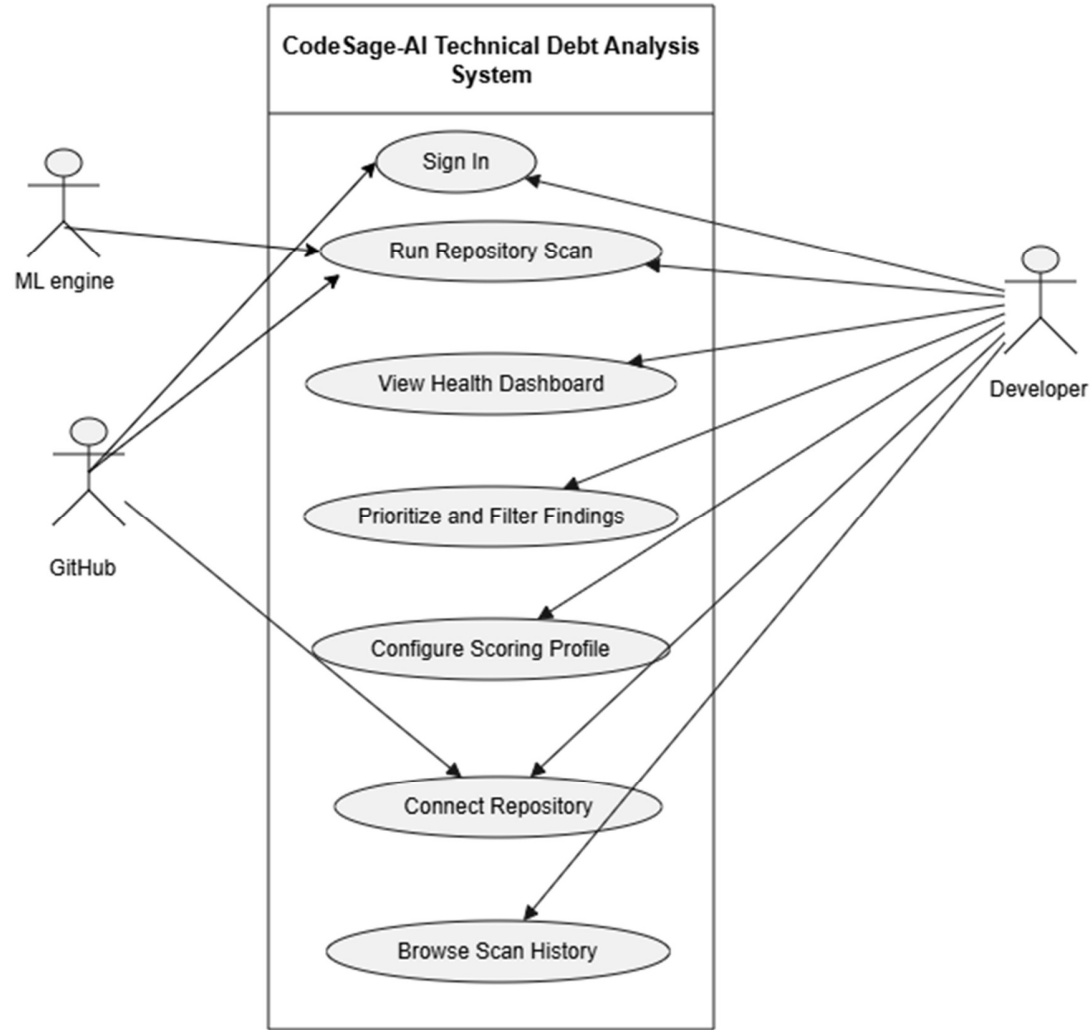
Table 4-1. Architecturally significant use cases.

ID	Use case	Actors	Why it is architecturally significant
UC-1	Sign In / GitHub Authentication	Developer	Establishes authenticated access to the system and provides the security foundation for repository access and role-based features.
UC-2	Connect Repository	Developer GitHub Service	Establishes the GitHub integration and repository context required for subsequent code analysis and scanning.
UC-3	Run Repository Scan	Developer	Core analysis use case. Initiates the asynchronous analysis pipeline involving repository extraction, code metrics, SATD detection, risk prediction, scoring, and persistence.
UC-4	View Health Dashboard	Developer	Aggregates analysis results into repository health, technical-debt indicators, risk information, and visual summaries for architectural monitoring.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

UC-5	Triage & Filter Findings	Developer	Provides mechanisms to inspect and prioritize detected technical-debt findings, connecting analysis results with actionable developer decisions.
UC-6	Select / Configure Scoring Profile	Developer	Allows users to configure category weighting used by the deterministic scoring engine, making prioritization adaptable to project needs.
UC-7	Browse Scan History	Developer	Provides access to previous scan results and snapshots, supporting historical comparison and tracking of technical-debt evolution.

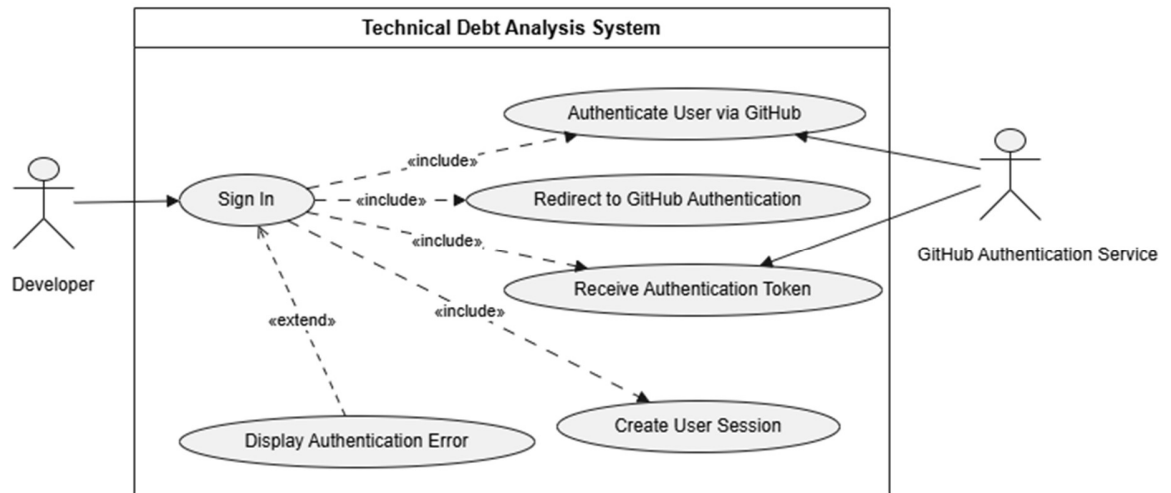
Figure 1. Use-case overview for v1.0.



4.1 Sign In

Figure 1.1. Use-case sign In.

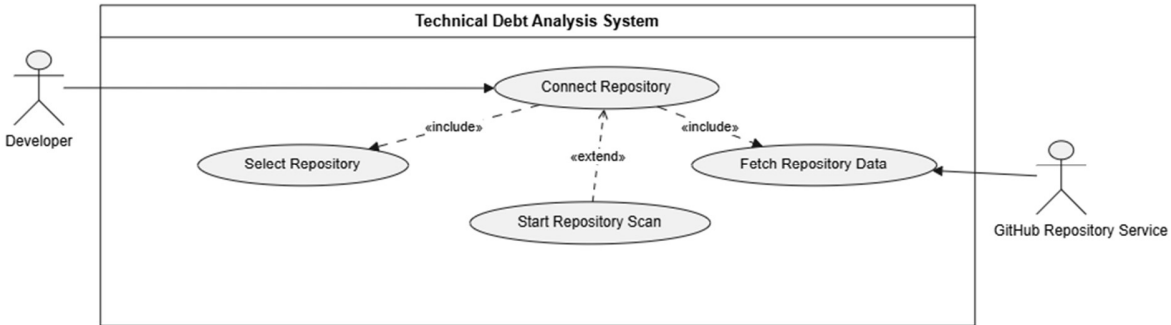
Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	



The Sign In process begins when a developer initiates authentication and the system sends the user to the GitHub Authentication Service, where the credentials are checked for verification by GitHub. Once they are verified, the system obtains an authentication token and confirms credentials and create user session If there is a failure in the authentication process or if the authentication token has failed, then the system will display authentication error.

4.2 Connect Repository

Figure 1.2. Use-case Connect repository.

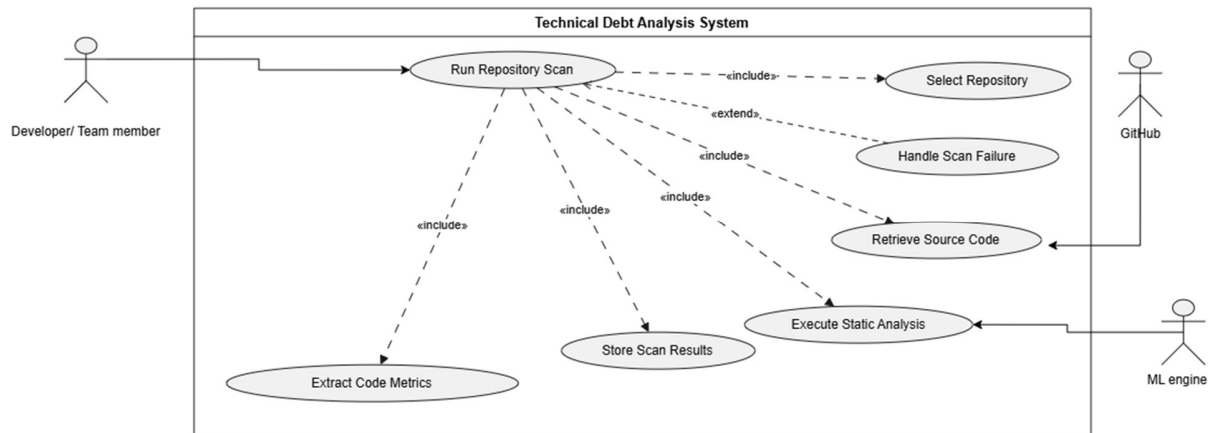


The Connect Repository process begins when a developer inputs or selects a public repository, causing the system to retrieve the metadata and the source files directly from the GitHub Repository Service into the workspace. Once the repository is connected and loaded successfully, the developer may start the scan.

4.3 Run Repository Scan

Figure 1.3. Use-case run repository scan.

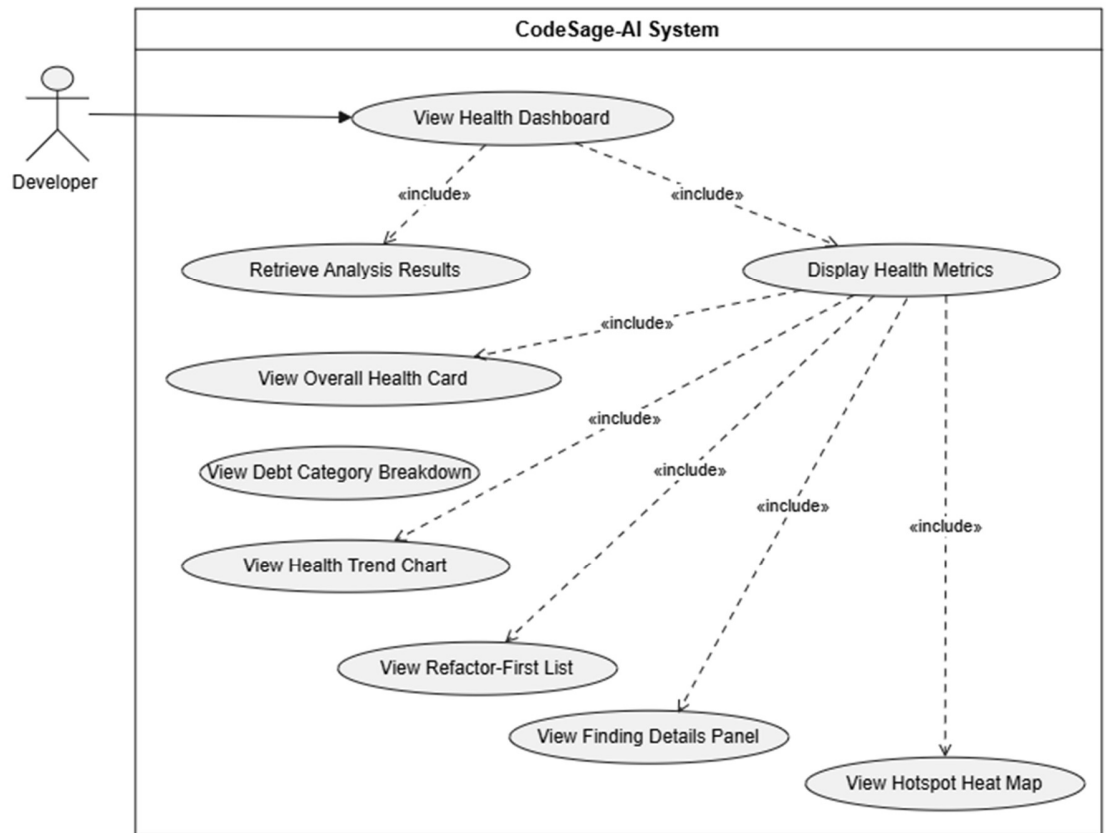
Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	



The process of running repository scanning begins the moment the developer chooses a GitHub repository. Once chosen, the system automatically downloads the code from GitHub and sends it to the ML Engine to run static analysis rules. In this process, the system Extract Code Metrics and saves the complete report in the database for the developer to review. If there is an error during the downloading or processing, the system handles it and notify the user.

4.4 View Health Dashboard

Figure 1.4. Use-case View health dashboard.



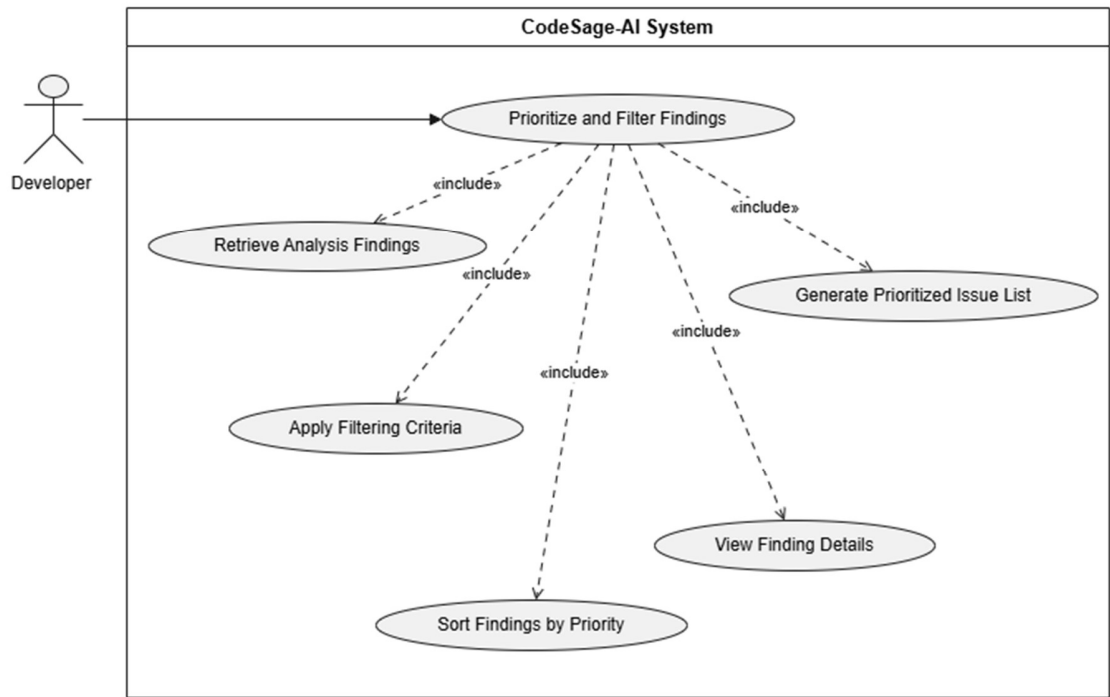
The process of viewing the health dashboard starts when a developer accesses the dashboard, prompting the system

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

to automatically retrieve analysis results and displaying the health metrics. Once loaded, the application automatically generates a report providing detailed information, metrics included such as overall score, categorization of technical debt, historical trends charts, list of prioritized refactoring lists, finding details, and a hotspot heat map for the developer to inspect.

4.5 Triage & Filter Findings

Figure 1.5.. Use-case prioritize and filter.

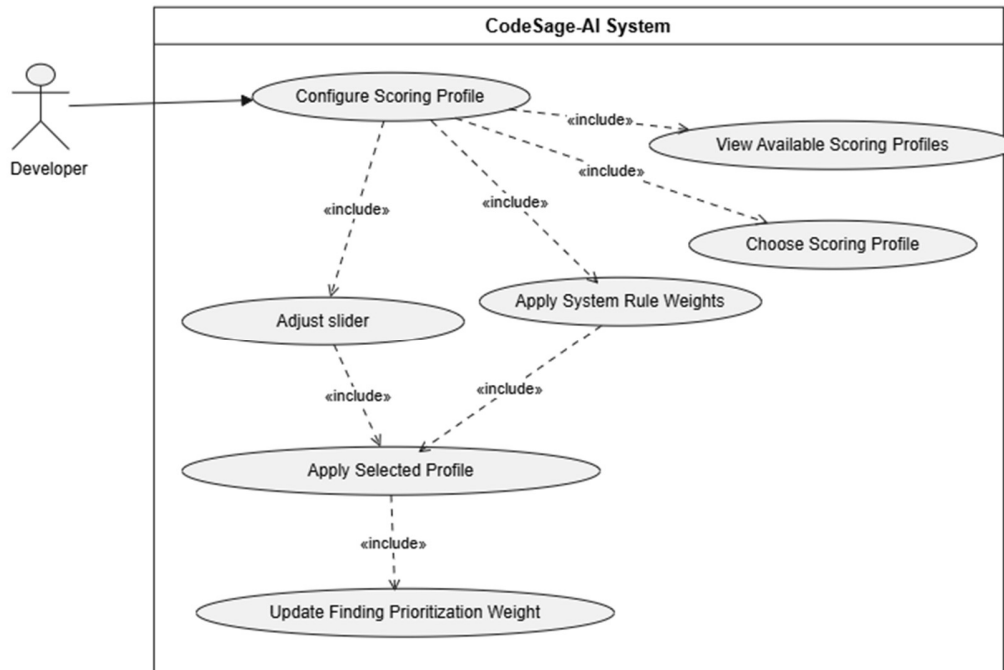


The process of prioritization and filtering starts whenever a developer asks to organize the scan results. Then the system retrieves previously stored findings and uses the selected filtering and sorting parameters. After that, the system processes the information to create a priority list of technical debt.

4.6 Select / Configure Scoring Profile

Figure 1.6. Use-case configure scoring profile.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

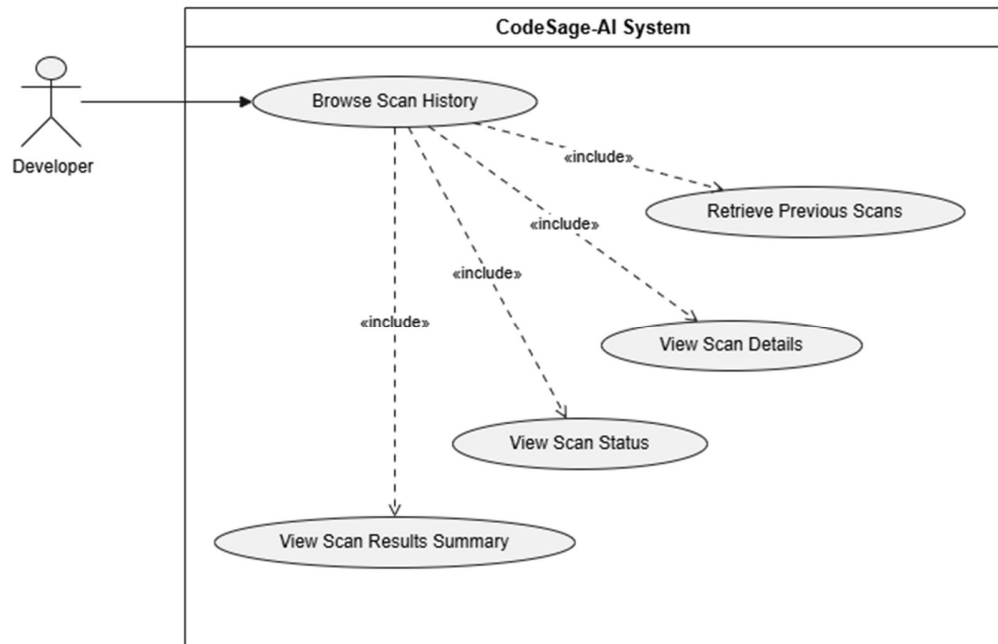


The Configure Scoring Profile process begins when a developer reviews available options and choosing a scoring profile. After that the developer customizes the parameters using sliders while the system uses the analysis logic that has been pre-defined Finally, the system combines these configurations into one and automatically updates the refactor first list.

4.7 Browse Scan History

Figure 1.7. Use-case browse scan profile.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	



The Browse Scan History process allows a developer review past static analysis activity by fetching previous analysis records from the database. When the data loads the system provides the execution states for every instance and a summary of the outputs. The Developer can then choose any past record to examine the execution log and breakdown in more details.

5. Logical View

5.1 Overview

Table 5-1. Subsystem decomposition.

Subsystem	Responsibility	Main internal parts
Frontend (Next.js)	Provides a browser-based user interface for managing repositories, controlling scans, dashboard, inspecting findings, scan history and scoring configurations. It communicates with the backend through the application API.	Next.js presentation components, dashboard views, project and branch selection, scan controls, finding-detail view, hotspot file tree, scan-history view, scoring-profile interface, API client.
Backend (FastAPI and Celery)	Implements the application and domain logic. FastAPI handles synchronous user-facing operations, while Celery workers perform long-running repository analyses asynchronously. The backend also derives profile-dependent scoring results from stored analysis facts.	FastAPI application services, repository and GitHub integration, analysis orchestration, Celery workers, metric and comment extraction, rule engine, scoring engine, snapshot management, dashboard services, security auditing.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Subsystem	Responsibility	Main internal parts
ML Service (Python, scikit-learn)	Performs machine-learning inference for SATD classification and file-level bug-proneness prediction. It returns prediction results to the analysis pipeline without assigning finding severity.	SATD classifier, bug-risk model, versioned model artifacts, model-version metadata.
Database (PostgreSQL)	Persists tenant-owned configuration, repository metadata, analysis history, immutable completed-analysis facts, findings, metrics, and prediction results. Profile-dependent scores are derived when results are retrieved rather than stored.	Workspace and repository data, analysis attempts, snapshots, findings, metrics, prediction data, scoring configuration, model provenance, security-audit records.

Figure 2. Logical decomposition into subsystems and layers.

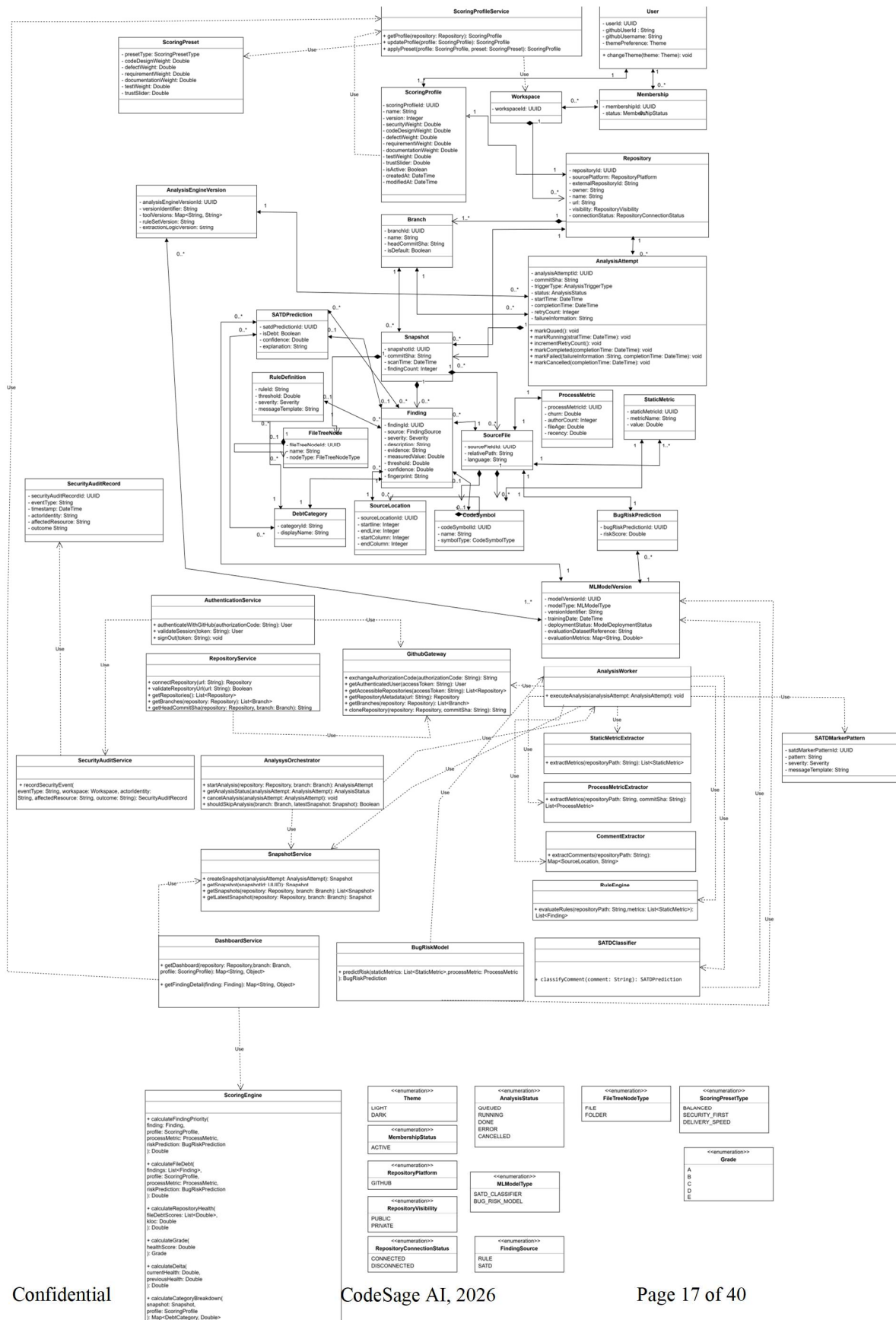
5.2 Architecturally Significant Design Packages

The system is organized into a set of architecturally significant design packages, each grouping closely related classes according to their primary responsibilities. These packages define the major logical boundaries of the system and support separation of concerns, maintainability, and modular development. The key design packages, their classes, and their roles are summarized in the following table.

Design Package	Classes Belonging to the Package	Role of the Package
Identity and Workspace Management	User, Workspace, Membership, AuthenticationService	Manages authenticated users, workspace boundaries, and user-to-workspace membership. It establishes the tenant context used to isolate repositories, analyses, and other workspace-owned data.
Repository Integration and Management	Repository, Branch, RepositoryService, GitHubGateway	Manages repositories and branches connected to a workspace and encapsulates interaction with GitHub. It retrieves repository metadata, branch information, authentication-related repository access, and repository content required for analysis.
Analysis Execution and Snapshot Management	AnalysisAttempt, Snapshot, AnalysisEngineVersion, AnalysisOrchestrator, AnalysisWorker, SnapshotService	Manages the complete lifecycle of repository analysis. It records individual analysis attempts, coordinates asynchronous analysis execution, identifies the analysis-engine configuration used, and creates immutable snapshots only after successful completion of the full analysis pipeline.
Metric and Source Information Extraction	StaticMetricExtractor, ProcessMetricExtractor, CommentExtractor	Extracts the source and repository information required by downstream analysis stages. This includes static software metrics, repository-history-derived process metrics, and source-code comments used for SATD detection.
Deterministic Technical Debt Detection	Finding, RuleDefinition, SATDMarkerPattern,	Represents and detects deterministic technical-debt findings. It maintains rule definitions and SATD severity

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

	RuleEngine	marker patterns and applies deterministic rules to metrics and source information to produce categorized findings with fixed severity.
Machine Learning Analysis	SATDPrediction, BugRiskPrediction, MLModelVersion, SATDClassifier, BugRiskModel	Encapsulates machine-learning-based analysis. It performs SATD classification and file-level bug-risk prediction while preserving the model-version information required for prediction provenance and reproducibility.
Scoring and Prioritization	ScoringProfile, ScoringPreset, ScoringEngine, ScoringProfileService	Defines and manages configurable scoring behavior and derives finding priorities, file-level debt, repository health, grades, category breakdowns, and related values from stored analysis facts. Scoring is performed on the read path so profile changes can re-score existing results without requiring another repository analysis.
Dashboard and Reporting	DashboardService	Assembles stored analysis facts and dynamically derived scores into read-only representations required by the dashboard, including current and historical repository health and technical-debt information.
Security Auditing	SecurityAuditService, SecurityAuditRecord	Records and represents security-relevant system events. It provides an auditable history of actions and outcomes without persisting authentication secrets.



Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Figure 3. Core domain model.

Table 5-2. Architecturally significant classes.

Class	Responsibility	Notes
User	Represents an authenticated user of the platform.	User identity is kept separate from workspace membership.
Workspace	Represents the tenant boundary and top-level owner of system data.	A workspace has one active user in v1.0, while the model supports future multi-user workspaces.
Membership	Represents the relationship between a user and a workspace.	Supports tenant membership without directly coupling users to repositories.
Repository	Represents a GitHub repository connected to the system for analysis.	Stores repository metadata and belongs to a workspace.
Branch	Represents a repository branch and its current head revision.	Analysis and historical trends are per branch
Analysis Attempt	Represents one execution attempt of the repository-analysis pipeline.	Failed and cancelled attempts are retained; only successful attempts produce finalized snapshots.
Snapshot	Represents the immutable result of a successfully completed analysis.	Stores analysis facts only. scores are derived when results are read depending on the profile.
Finding	Represents an atomic technical-debt issue detected in source code	A finding has source rule or satd; category and severity are fixed at detection time.
RuleDefinition	Defines a deterministic technical debt detection rule.	Each rule has a fixed category, severity, threshold, and message template.
SATDMarkerPattern	Defines deterministic marker patterns used to assign severity to SATD findings.	SATD severity is assigned from comment patterns rather than by the ML model.
SATDPrediction	Represents the output of SATD classification for a source-code comment.	Records debt/non-debt classification and category-related prediction information.
BugRiskPrediction	Represents the file-level bug-proneness result produced by the risk model.	Produces a risk score but does not create a technical-debt finding.
MLModelVersion	Represents version and evaluation metadata for a deployed machine-learning model.	Supports model provenance and reproducibility across analyses.
AnalysisEngineVersion	Represents the exact analysis-engine configuration used for an analysis attempt.	Identifies tool, rule-set, extraction-logic, and ML-model versions.
ScoringProfile	Represents the configurable values used to derive finding priorities and repository health.	Contains category weights and trust configuration.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Class	Responsibility	Notes
ScoringPreset	Represents a predefined scoring-profile configuration.	Used to initialize or reset profile values.
AuthenticationService	Handles GitHub authentication, session validation, and sign-out behavior.	Stateless application service.
RepositoryService	Handles repository connection and retrieval of repository and branch information.	Uses the GitHub gateway for external repository access.
GitHubGateway	Encapsulates all communication with GitHub.	Handles OAuth, repository metadata retrieval, branch information, and repository cloning.
AnalysisOrchestrator	Controls the lifecycle of asynchronous repository-analysis requests.	Starts, monitors, cancels, and may skip equivalent analyses.
AnalysisWorker	Executes the repository-analysis pipeline asynchronously.	Coordinates extraction, deterministic detection, ML inference, and snapshot finalization.
StaticMetricExtractor	Extracts static software metrics from the checked-out repository revision.	Produces metrics used by rule evaluation and bug-risk prediction.
ProcessMetricExtractor	Extracts history-derived process metrics from the repository.	Produces churn, author count, file age, and recency values.
CommentExtractor	Extracts source-code comments and their locations for SATD analysis.	Unrelated comments remain transient and are not persisted.
RuleEngine	Evaluates deterministic metric thresholds and security patterns to produce rule-based findings.	Security findings are still rule findings with category security.
SATDClassifier	Classifies source-code comments as debt/non-debt and predicts SATD category.	Does not assign severity.
BugRiskModel	Predicts file-level bug-proneness from static and process metrics.	Its output influences scoring but does not generate findings.
ScoringEngine	Derives finding priority, file debt, repository health, grade, delta, and category breakdown.	Operates on the read path as a pure calculation over stored facts and the active profile
SnapshotService	Creates finalized snapshots and retrieves current or historical snapshots.	Finalization occurs only after all analysis stages complete successfully.
DashboardService	Assembles dashboard data from stored analysis facts and derived scores.	Provides read-only dashboard representations.
ScoringProfileService	Retrieves and updates scoring profiles and applies scoring presets.	Profile changes re-score existing results without triggering a new scan.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Class	Responsibility	Notes
SecurityAuditService	Records security-relevant system events.	Produces persistent audit records without storing authentication secrets.
SecurityAuditRecord	Represents a persisted security-audit event.	Stores event type, timestamp, actor identity, affected resource, and outcome.

6. Process View

The processes. Five processes make up a running system. The **API process** is stateless and can be replicated. One or more **worker processes** run the scan pipeline. The **ML inference process** loads the two model artifacts and answers classification and risk requests. The **Redis broker** carries job messages, progress and the cancel flag. The **database process** holds everything that persists.

Table 6-1. Processes and their communication.

Process	Characteristics	Communicates to	Carries
API (FastAPI)	Stateless (request/response style)	Redis, PostgreSQL, GitHub	HTTP requests, Job enqueue, scoring
Worker (Celery)	Long running, background	Redis, PostgreSQL, ML service, GitHub	Clone → extract → detect → Score pipeline
ML inference	Long running, (request/response style)	Called by workers	Comment batches in, labels out; feature vectors in, risk scores out
Redis broker	Transient state, in-memory	API and workers	Job queue, progress percentage, cancel flag
PostgreSQL	Stateful	API and workers	All persistent state

Figure 4 draws above five processes and adds two more. The browser appears because a scan is started and polled from there. GitHub appears because both the API and the worker call it. Neither is a process this project deploys, but messages cross to both, and omitting them would hide two of the system's interfaces.

Figure 4 traces one scan from the user's click to the rendered dashboard. Each process has its own region, so every arrow that crosses boundary is a message between two processes, and the label on that arrow names the protocol it travels over. GitHub is drawn as a seventh process(external) because both the API and the worker call it, in two different ways. The scan phase defined in SRS FR-6 is written on the activities that change it, so the state machine idle → queued → running NN% → done | error | cancelled can be followed through the figure.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

a worker, and cloning a repository only to discover that nothing had changed.

The comparison uses the last successful scan, not the last scan. A cancelled or failed scan leaves a SCAN row that has no FINDING and no FILE_SCORE rows, because the worker stopped before the persist stage. If the head SHA were compared against such a row, the system would skip the work and serve a snapshot that was never written. The same qualifier applies to the dashboard read, which resolves the latest snapshot where phase = done.

GitHub is contacted in two different ways. The API uses the GitHub REST API for repository and branch metadata, with ETag conditional requests so that repeated reads usually cost no rate-limit quota. The worker uses git clone over HTTPS to obtain the code, which consumes no REST quota at all. This is why the rate limits are avoided rather than managed

The API answers before the work begins. POST /api/repos/{repoId}/scan inserts the SCAN row, enqueues the job and returns a scan identifier with phase = queued. The client then polls GET /api/repos/{repoId}/scan/{scanId} once per second (SRS Table 3.97). The interface stays usable while the scan runs, as SRS PERF-05 requires. Running the pipeline inside the request would block the connection for minutes and make the requirements impossible to meet.

PostgreSQL holds the phase and Redis holds only the percentage. The status endpoint reads SCAN.phase from the database and the progress percentage from Redis. The separation follows from what each store guarantees. Redis is a broker, so losing a percentage when it restarts costs nothing, because the next poll produces a new one. Losing the fact that a scan failed would break SRS SP-13, which requires the final phase and its error message to be recoverable from the database alone. Every terminal phase is therefore written to SCAN by the process that reaches it.

Cancellation is cooperative rather than forced. The API does not stop the worker. It sets a flag in Redis and returns immediately. The worker reads that flag between stages and stops at the first boundary it reaches, deletes its clone and writes phase = cancelled. Storing the snapshot is outside this region: once the worker begins writing findings it completes the write. Terminating the process during a write would leave a partial snapshot, and SRS FR-6 requires the previous snapshot to remain intact after a cancellation. The cost of this choice is response time, because a user who presses Stop waits until the current stage ends.

The cancel result reaches the user through the polling channel. Because the worker writes phase = cancelled to the same row the status endpoint already reads, no separate notification path is needed. The user learns that the scan has really stopped when the next poll returns idle. The three non-running phases, done, error and idle, all arrive by the same route.

A failed scan is recorded in the database, not only in the logs. The worker writes phase = error and the error message onto the existing SCAN row. This satisfies SRS SP-13: a failure reported by a user can be diagnosed from the database without reading server logs. Nothing was written to FINDING or FILE_SCORE, so the previous snapshot is unaffected and remains what the dashboard shows.

The two machine-learning models are independent and are not chained. ML-1 and ML-2 take different inputs, produce different outputs and exchange no data. ML-1 reads the source comments and produces SATD findings, predicting category only. It can produce four of the five categories; security comes from the rule engine alone (SRS FR-9.3). Severity for a SATD finding comes from the deterministic marker table in SRS Appendix C.2, not from the model (SRS FR-9.2). ML-2 reads the CK product metrics and the four PyDriller process metrics and produces one risk_score per file. It produces no findings at all (SRS FR-10).

The machine-learning models can be skipped. Both models run in one inference container, so they are reachable or unreachable together. The dashed edge that bypasses the lane leaves the worker, because the worker is where the call is attempted and therefore where the failure is observed. When the service is unreachable the worker persists a valid snapshot: all rule and security findings are present, no SATD findings appear, and every risk factor falls back to 1.0 so that no finding receives ML-2 risk scores.

6.1 The extraction boundary

Git history enters the pipeline as numbers, never as text. Text produces findings, and a finding must map to a file: line that the user can open. So, text is read from the checked-out tree at the scanned commit. History produces metrics for the past 90-days, but numeric values enter the scanning pipeline.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Table 6-2. What may and may not enter the pipeline.

Input	Used at scan time (Yes/No)	Reason
Source comments at the scanned SHA	Yes	The SATD classifier reads comments
Source files at the scanned SHA	Yes	CK, then the rule engine and ML-2 use source files
Commit history reachable from that SHA	Yes	Four numbers: churn, author count, file age, recency
Commit message text	No	It has no file: line, so a finding could not point anywhere
Pull requests, issues, GitHub API metadata	No	Pull requests and issues not considered in v1.0. Api quota not used during a scan
Previously stored snapshots	No	History is read only to draw trends and deltas, never to detect or score

Two consequences follow.

A scan is a pure function of the repository at one commit. The result depends on the tree and reachable history at that SHA plus a fixed model version, never on a previous scan row. This is what makes the skip branch of decision in Figure 4 safe, and what makes snapshots reproducible. Since, a scan is independent of prior scans, not of git history, because ML-2 reads that history as numbers.

The churn window is anchored to the commit, not to the clock. The 90 days are measured backwards from the scanned commit's committer date, never from scanned time.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

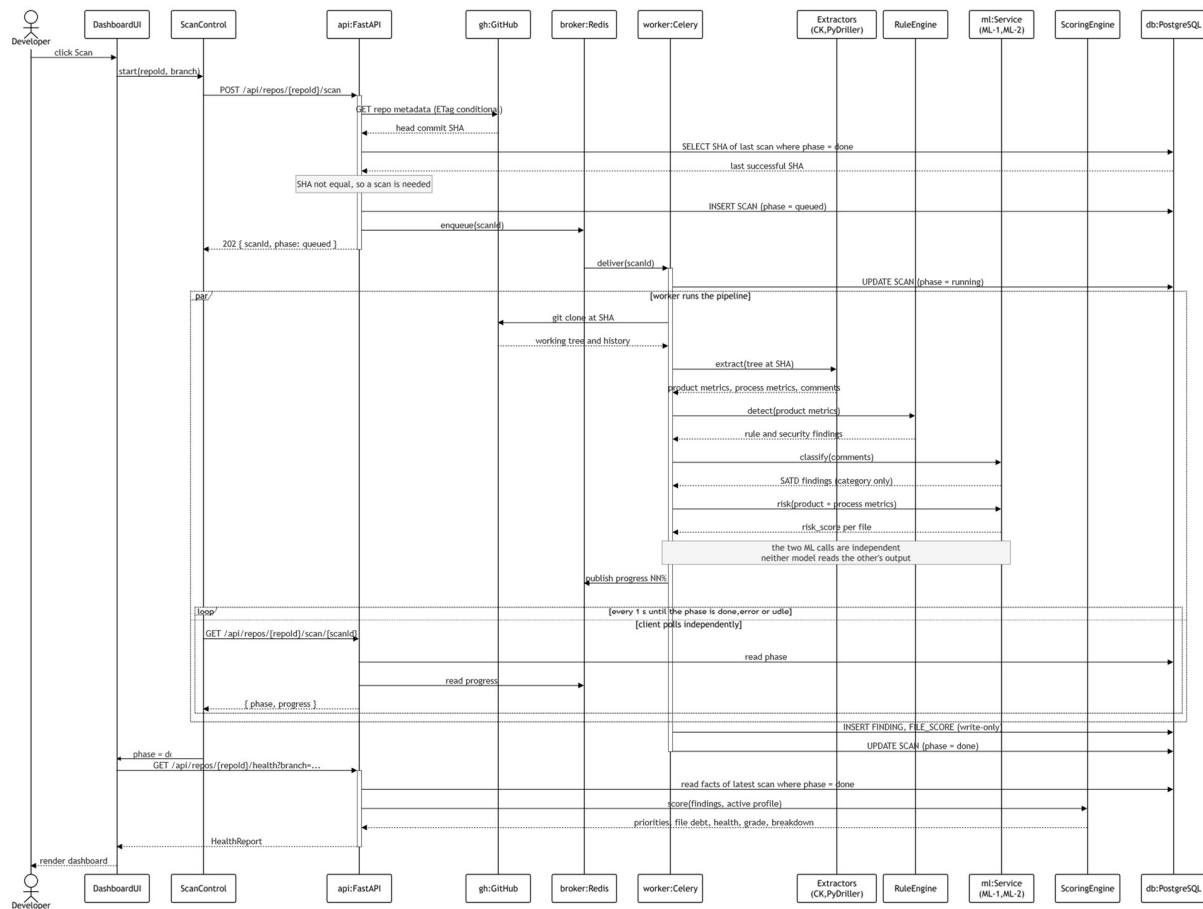


Figure 5. Run a scan sequence, successful path

Figure 5 shows the same scan as Figure 4, but as an ordered exchange between the objects that carry it out. Where Figure 4 answers "which process does what", Figure 5 answers "in what order, and who waits for whom".

The figure shows the successful path only. Cancellation and failure are drawn in Figure 4 instead. A sequence diagram states one scenario well, and adding two mutually exclusive endings to it would make the main path harder to read rather than easier. Figure 4 is the right place for alternative flows because an activity diagram is built around branching.

The ordering carries the decisions below.

The API consults GitHub and the database before it queues anything. It reads the branch head SHA from GitHub, then reads the SHA of the last scan whose phase is done. Only when the two differ does it insert a SCAN row and enqueue a job. Comparing against the last successful scan is what stops a cancelled scan from being mistaken for a stored snapshot.

The API replies before the work starts. The 202 carrying the scan identifier is returned as soon as the job is on the queue. The activation bar on the API ends there. Everything after that point happens without a client connection being held open, which is what SRS PERF-05 requires.

The worker never calls the API. It records its phase by writing to SCAN and its progress by publishing to Redis. The API serves the polling client from those two sources. This keeps the dependency direction one way and matches the deployment view in Figure 7.

The polling loop and the pipeline are concurrent. They are drawn as parallel fragment because the client polls while the worker works. Placing the loop before the worker starts would describe a synchronous system and would contradict the design.

The status endpoint reads from two places. The phase comes from PostgreSQL and the percentage from Redis. A

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

lost percentage can be recomputed by the next poll, whereas a lost failure would break SRS SP-13.

The two model calls are independent. They are drawn as two separate pairs, not as a chain. Neither model reads the other's output, so an implementation may issue them together.

The two client objects have separate responsibilities. ScanControl owns the scan lifecycle and stops when the poll returns done. It then hands over to DashboardUI, which issues the dashboard read and renders the result. Keeping the request and the render on the same object means every reply returns to the object that asked for it.

Finally, ScoringEngine is invoked by the API, after the worker has finished and the client has asked for the dashboard. It is never invoked by the worker.

6.2 Applying a scoring profile

A profile change is the only other user initiated write in v1.0, and it is deliberately the opposite of a scan in every way. Nothing is queued, no worker wakes and no snapshot row is written (SRS FR-20, FR-21).

The full workflow, step by step.

The Profiles screen loads. It reads GET /api/profiles/active and sets the sliders to the values that are currently set.

The user picks one of the three presets, or drags the five weight sliders and the trust slider directly. Balanced is the workspace default. Nothing is sent to the server while the user drags. The values live in the browser only.

The user clicks Apply.

The client sends PUT /api/profiles/active. The body carries the complete profile, which is the five weights and trust_s.

The server validates the values and clamps them. Weights are clamped to the range 0.1 to 3.0, and trust_s to the range 0 to 1.

The server writes both rows in one transaction. It updates weights and trust_s on SCORE_PROFILE, and sets active_profile_id on WORKSPACE. Seven numbers change. That is the whole write.

The server returns the profile as it was stored, after clamping. The screen therefore shows the values that are really in force, not the values the client sent.

The client re-reads GET /api/repos/{repoId}/health?branch; the same URL it always uses. No profile appears in it.

The server reads the stored findings and the per-file facts, passes them to ScoringEngine, and returns the re-scored dashboard.

Both endpoints are standard. SRS Table 3.96 defines them, so this section describes an agreed interface rather than proposing a new one. GET /api/profiles/active returns the workspace's active profile and seeds the screen when it loads, while PUT /api/profiles/active applies a profile the user has adjusted. FR-20 sets out the contract that both follow, requiring that a profile be applied in *"a single idempotent write carrying the complete profile"*, and that dragging a control *"changes client state only; no request is issued until the user confirms with Apply"*. The same requirement adds that the server *"clamps every weight on write and returns the stored profile"*, so that the client can confirm what was actually saved rather than assuming that the values it sent were accepted.

Why PUT and not PATCH. PUT carries the meaning "make this resource look exactly like this", so the body it takes is the complete profile rather than a description of what has changed within it. The practical benefit of that distinction is that retrying the request after a dropped response cannot apply the same adjustment twice, which matters here because the dependent read in step 8 follows the write immediately and would otherwise render a dashboard matching no profile the system holds.

Why profile is not in the URL. FR-20 settles the question directly, stating that *"the active profile is server-side state scoped to the workspace, so the read endpoints are unchanged and carry no profile parameter."* The architectural reasoning behind the requirement is straightforward, because if the profile travels in the query string, every read endpoint would gain a parameter each time the profile changed shape, and the scoring formula would gradually leak into the API surface. SRS Table 3.96 shows the outcome of that decision, since GET /api/repos/{repoId}/health?branch= takes a branch and nothing further, deriving its scores under whichever profile is active at the moment the request is served.

Table 6-3. The two write paths compared.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Aspect	Run a scan	Apply a profile
Request	POST /api/repos/{repoId}/scan	PUT /api/profiles/active
Handled by	API → Redis → Celery worker	The API process alone
Writes	A new immutable SCAN with its FINDING and FILE_SCORE rows	Seven numbers on one SCORE_PROFILE row
Duration	Seconds to minutes, polled	One round trip
Effect on findings	Creates them	Leaves them completely untouched

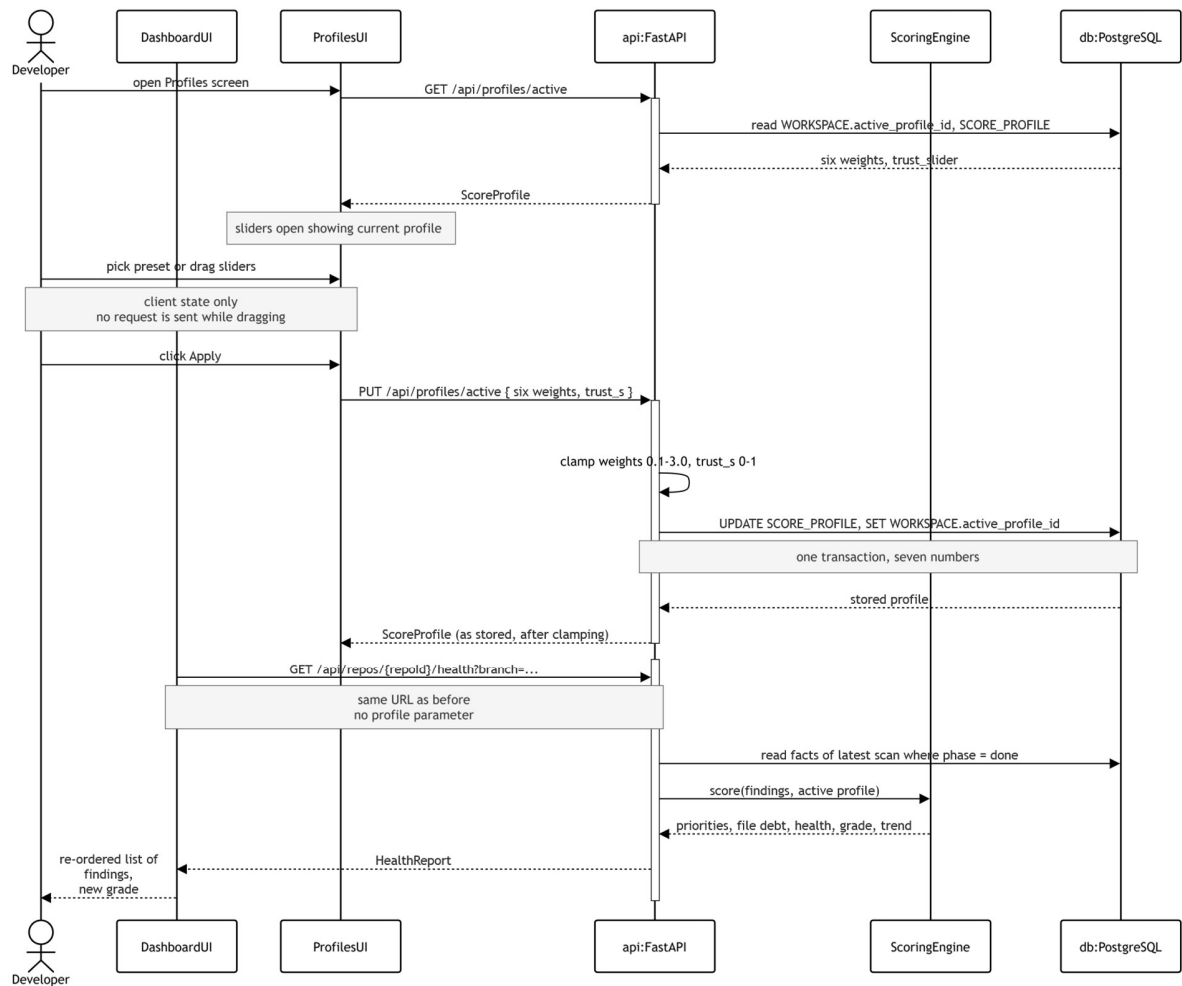


Figure 6. Apply a profile sequence.

The decisions recorded in the figure are set out below.

Nothing is sent to the server while the user drags. The slider positions remain client state until Apply is pressed, which is what SRS FR-20 requires. Issuing a request on every slider movement would place a write on the server for each pixel of travel, and each of those writes would in turn have to be validated, clamped, stored and read back before the interface could display a confirmed value.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

The write is a single transaction covering seven numbers. The five category weights and the trust slider are written to SCORE_PROFILE at the same moment that active_profile_id is set on WORKSPACE, so that no reader can ever observe one without the other. No SCAN, FINDING or FILE_SCORE row is touched anywhere in the exchange, which is precisely what SRS FR-21 means when it states that changing a profile creates no snapshot.

The server sends back the final saved settings after adjusting them to fit allowed limits. This lets the user see the true values used for their scores. Without this, the app might falsely show an unadjusted input (like 5.0) while the server actually uses a limited value (like 3.0)

The dependent read is issued against the same URL as before. GET /api/repos/{repoId}/health?branch=; carries no profile parameter, for the reason given earlier in this section: the active profile is server-side state belonging to the workspace (SRS FR-20).

ProfilesUI performs the write and DashboardUI performs the read. The two objects are drawn separately for the same reason they are separated in Figure 5, namely that the object issuing a request should also be the object receiving the reply. ProfilesUI signals that a profile has been applied, and DashboardUI then carries out the read it already owns.

Three properties follow from this shape.

The read endpoints remain unparameterised, so the API surface does not grow as the profile grows.

The PUT is idempotent by construction, because its body carries the complete profile rather than a description of a change to it. Retrying the request after a dropped response therefore cannot leave a partially applied profile, which matters because the client issues a dependent read immediately afterwards.

The profile is shared rather than held per tab. A reload, a second browser tab and a second member of the team all resolve to the same active profile, so the profile name displayed on the trend chart (SRS FR-14) always refers to something that genuinely exists on the server.

Clamping is a server-side rule rather than a client convenience (SRS FR-20). The client clamps its sliders so that the controls behave sensibly under the user's hand, but the server clamps for a different and stronger reason: repo_health is calibrated against the constant k, so a single unclamped weight arriving from any client would render every stored grade incomparable with every other.

6.3 The visibility floor

SRS FR-24 requires that a critical security finding remains visible however the active profile is set. Meeting that requirement is a responsibility of ScoringEngine, and the architecture satisfies it through three mechanisms acting together rather than through any single check, because one check placed somewhere in the scoring path could later be removed or reordered without anything else in the system appearing to fail.

Table 6-4. The three mechanisms of the visibility floor.

No.	Mechanism	Why it cannot be bypassed
1	Severity is assigned at detection and is not user-configurable	The rule register fixes a hardcoded secret at Critical and no profile control writes to FINDING.severity, so the value cannot be lowered from the interface at all.
2	Security findings bypass the trust slider	source_trust is held at 1.0 for the security category (SRS FR-11), so moving the slider alters rule_trust and ml_trust while leaving security findings entirely unaffected by it.
3	ScoringEngine pins critical security findings into the visible list	The pin overrides the computed order, so even the minimum security weight of 0.1 cannot push such a finding below the point at which the interface stops displaying it.

7. Deployment View

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

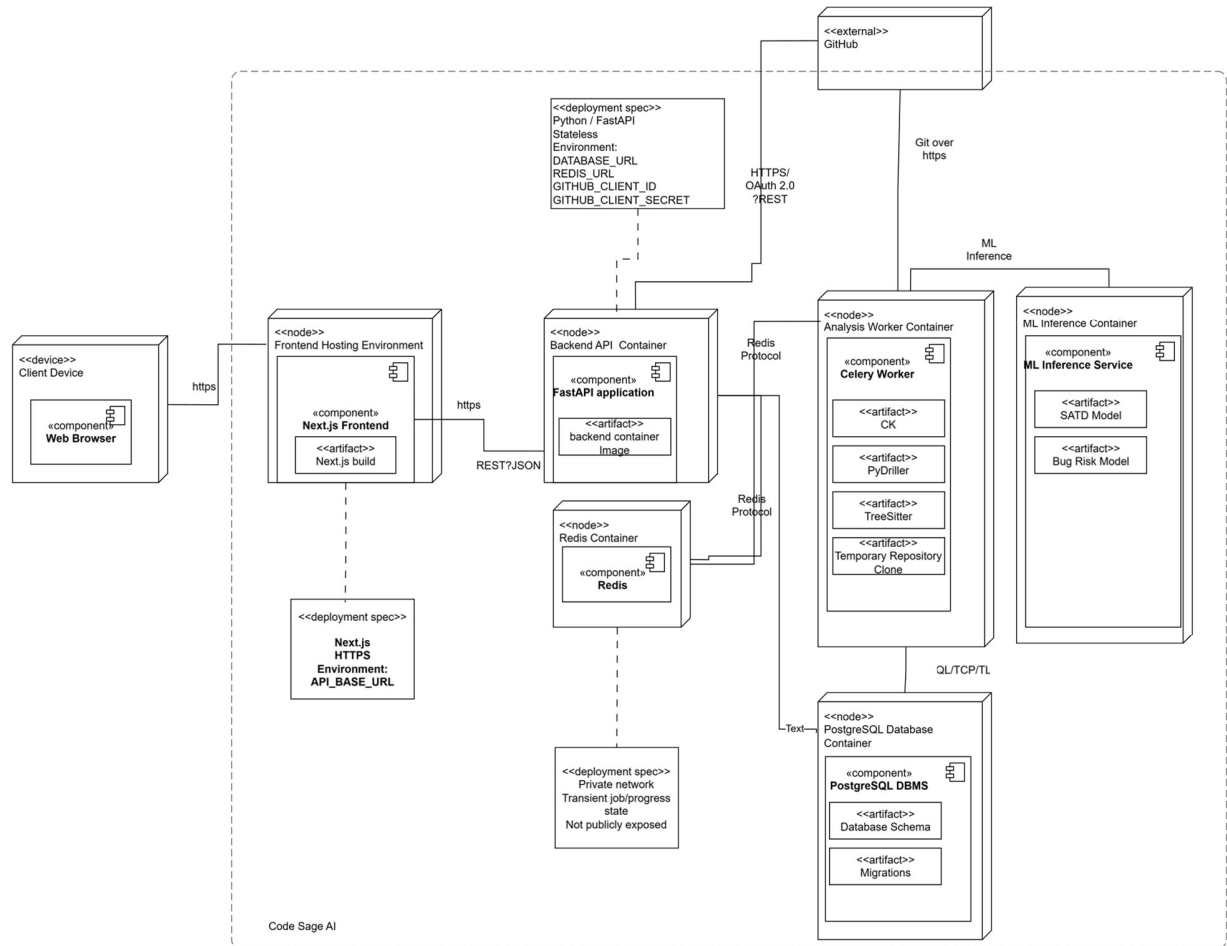


Figure 7. Deployment for v1.0.

Table 7-1. Physical nodes and the processes mapped onto them.

Node	Processes hosted	Connections
Client device	Web browser executing the client-side web application and rendering the Code Sage AI user interface.	Connects to the frontend over HTTPS. Makes application/API requests over HTTPS/JSON through the web application
Frontend container	Next.js frontend and deployed Next.js build. Handles presentation, navigation, dashboard rendering, user interactions, and API-client communication.	Receives HTTPS requests from the client browser. Communicates with the backend API using HTTPS/JSON REST.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Node	Processes hosted	Connections
Backend container	FastAPI application. Handles authentication, repository/project operations, branch retrieval, dashboard and analysis-history requests, scoring profiles, scan submission, status polling, and cancellation.	Communicates with the frontend through HTTPS/JSON REST; Redis for task enqueueing/progress; PostgreSQL for persistent data, GitHub through HTTPS/OAuth/REST as required.
Worker container(s)	Celery workers executing repository-analysis jobs; static analysis and repository-mining tools including CK, PyDriller, and Tree-sitter; temporary repository clones. Multiple instances may run concurrently.	Receives tasks through Redis; communicates with the ML inference service for predictions; accesses PostgreSQL for analysis state/results, clones repositories from GitHub using Git over HTTPS.
ML container	ML inference service, versioned SATD classification model and bug-risk prediction model.	Receives inference requests from analysis workers and returns SATD classifications/categories and file-level risk scores. Communication remains within the private network.
Redis broker	Redis message broker/task queue carrying analysis jobs and progress state.	Private connection between the backend API and Celery workers using the Redis protocol. Not publicly exposed.
PostgreSQL	PostgreSQL DBMS, application database schema, stored application/analysis data, and versioned database migrations.	Accessed by the backend API and analysis workers using SQL/PostgreSQL wire protocol over TLS through the private network.

8. Implementation View

8.1 Overview

Figure 8. Implementation layers and their components.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

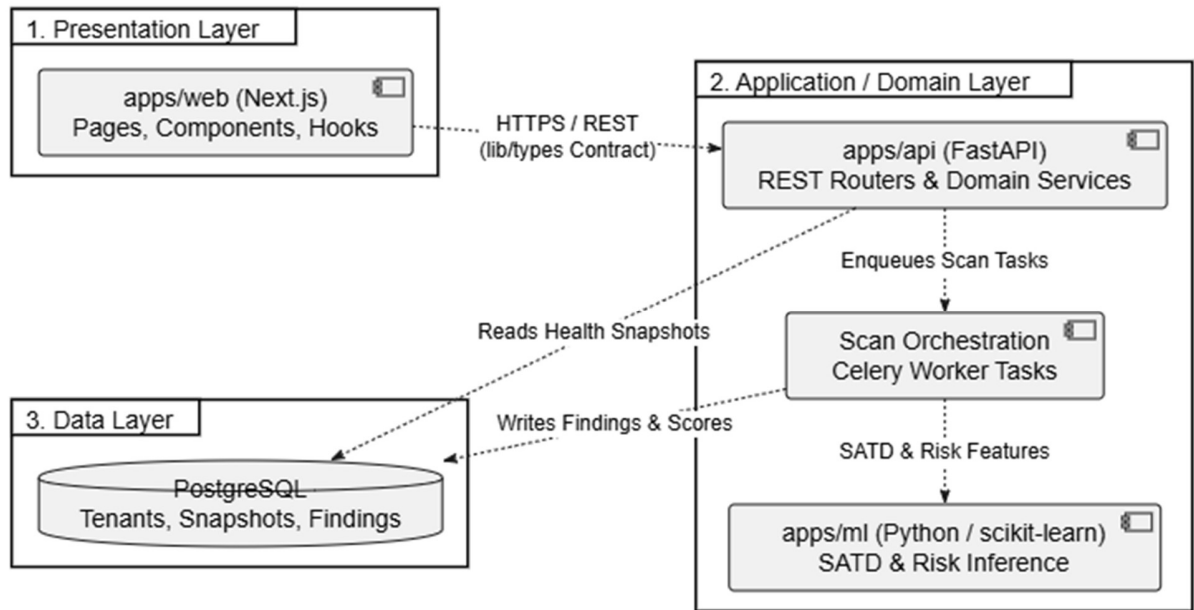


Table 8-1. The three layers.

Layer	What belongs in it	Rule for inclusion
Presentation	Next.js frontend, UI pages, dashboards, charts, repository/project management screens	Include components that directly handle user interaction and presentation of information.
Application / Domain	FastAPI API, analysis orchestration, SATD detection, technical-debt analysis, bug-proneness prediction, prioritization logic, Celery tasks	Include business logic, application services, use-case orchestration, and AI/analysis functionality.
Data	PostgreSQL database, database models/repositories, Redis, GitHub repository data access	Include components responsible for data persistence, retrieval, caching, and external data access.

8.2 Layers

Table 8-2. Rules that govern the layers.

Rule	What it requires
Volatile libraries sit behind one boundary each	External libraries and tools are accessed through dedicated interfaces or adapters.
Thresholds and weights are configuration, not code literals	Thresholds and scoring weights are stored in configuration rather than hard-coded.
Models are versioned artifacts loaded at runtime	ML models are versioned separately and loaded at runtime when required.
The presentation layer never reaches the database	The frontend accesses data only through the application/API layer and direct database access is forbidden.
Business calculations remain in the application layer	Scoring and prioritization calculations are performed by application logic, not in the database.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

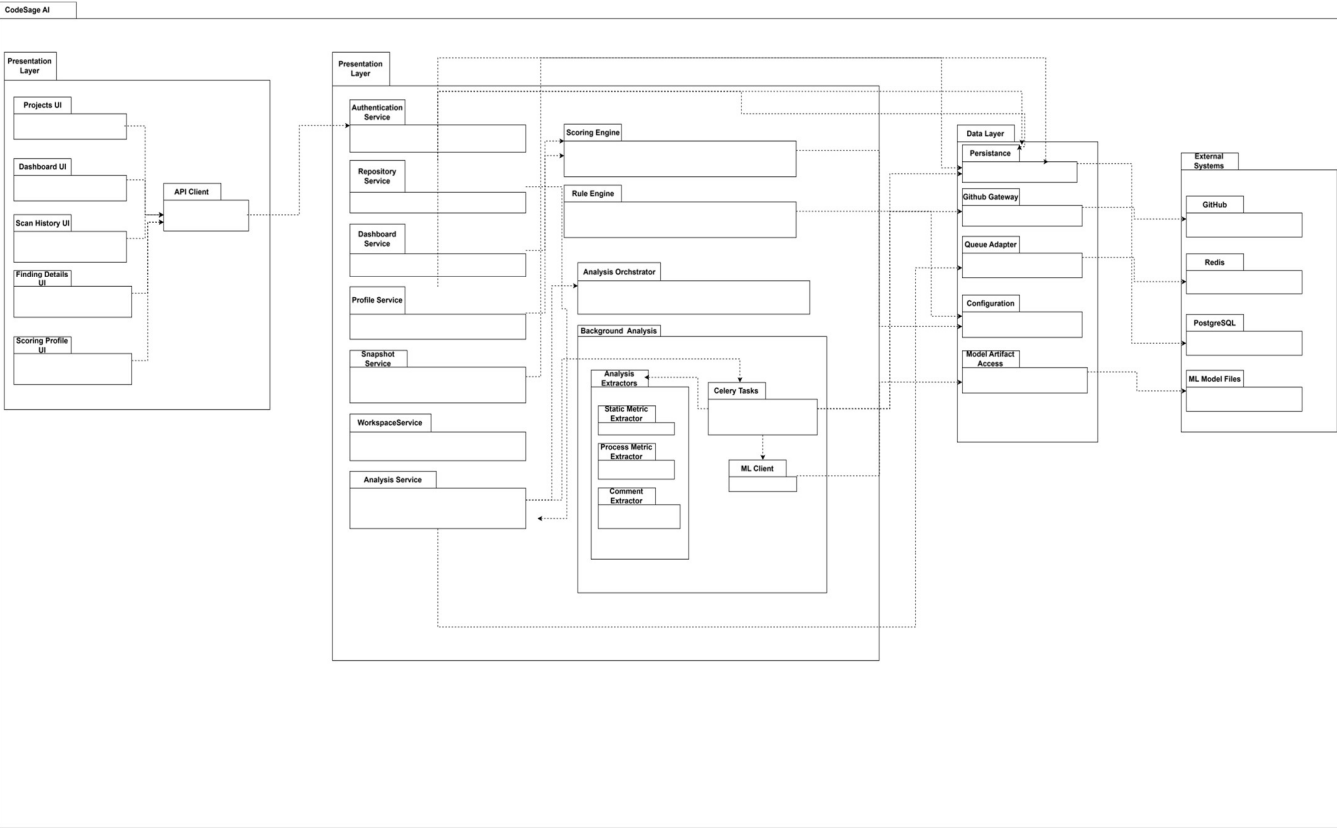


Figure 9. Package diagram of the backend application.

8.2.1 Presentation Layer

The Presentation Layer contains the Projects UI, Dashboard UI, Scan History UI, Finding Details UI, and Scoring Profile UI, which communicate with the backend through the API Client. It is responsible for user interaction and presentation while the database is forbidden.

8.2.2 Application Layer

The Application Layer contains the API routers and application services, including Authentication, Repository, Dashboard, Profile, Snapshot, Workspace, and Analysis Services, together with the Scoring Engine, Rule Engine, and Analysis Orchestrator. The background-analysis subsystem contains Celery Tasks, metric and comment extractors, and the ML Service. The scoring Engine is a pure Python function that takes stored findings, per-file facts, and the active profile and returns priorities, file debt, health, grade, and category breakdown, rather than computing these values in the database.

8.2.3 Data Layer

The Data Layer contains the Persistence, GitHub Gateway, Queue Adapter, Configuration, and Model Artifact Access packages. These components isolate database, GitHub, queue, configuration, and ML-model access from the application logic.

8.2.4 External Systems

The External Systems include GitHub, Redis, PostgreSQL, and ML model files, which provide external repository, queue, database, and model resources required by the system. Access to these resources is mediated through the respective adapters ensuring that external systems and databases are not accessed directly by the frontend.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

9. Data View

The diagram below represents the persistent data model of Code Sage AI. It is organized around four domains which are tenant and access management, repository analysis and snapshots, findings and prediction data and scoring and analysis configurations.

The workspace is the tenant boundary and it is associated with users through Membership. Repositories, branches, analysis attempts, snapshots, files, metrics and findings depict the persistent facts produced throughout the repository analysis lifecycle.

The supporting entities like MLModelVersion, AnalysisEngineVersion, RuleDefinition, DabtCategory, SATDMarkerPattern, Scoring Profile and ScoringPreset persist the configuration and provenance required to reproduce and interpret analysis results.

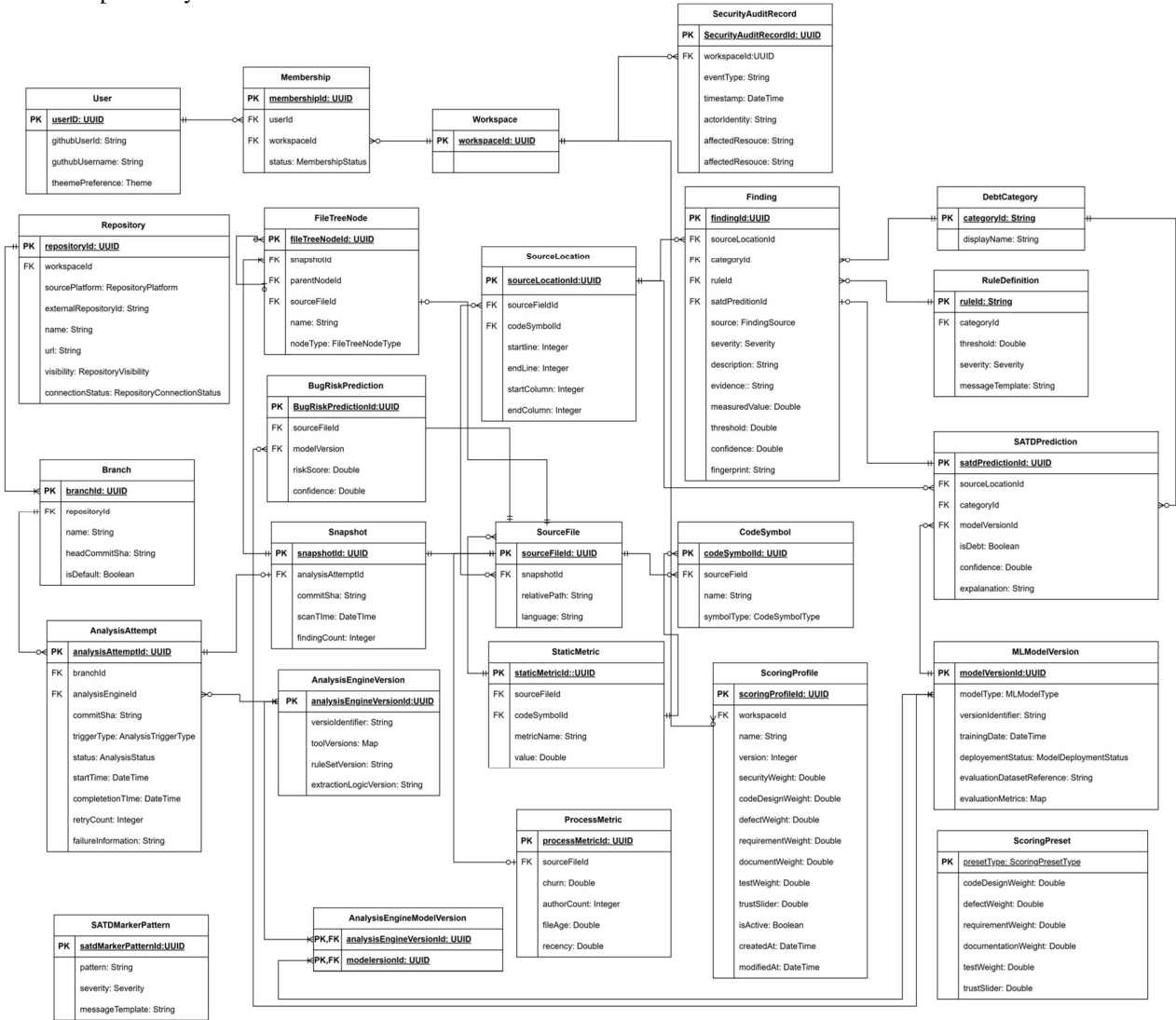


Figure 10. Multi-tenant data model.

The Data model can be viewed as four related groups.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Group	Entities	Role
Tenant and access data	User, Membership, Workspace, SecurityAuditRecord	Establishing user-to-workspace ownership, tenant isolation and auditable activities
Repository and analysis data	Repository, Branch, AnalysisAttempt, Snapshot, SourceFile, FileTreeNode, CodeSymbol, SourceLocation, StaticMetric, ProcessMetric	Describing the repository revision being analyzed and the facts extracted from it.
Findings and machine-learning data	Finding, DebtCategory, BugRiskPrediction, SATDPrediciton	Capturing detected technical debt evidence and machine learning outputs linked with the analyzed code.
Configurations and provenance data	RuleDefinition, SATDDMarkerPattern, ScoringProfile, ScoringPreset, MLModelVersion, AnalysisEngineVersion, AnalysisEngineModelVersion	Recording the rules, scoring configurations, models and analysis engine versions used to produce each result.

Table 9-1. Persistent entities.

Entity	What it holds	Stored fact or derived?
Workspace	Tenant boundary for repositories, memberships, scoring profiles, and other workspace-owned data.	Stored fact
Membership	GitHub-linked user information and the relationship between users and workspaces. 'membership' allows the data model to support multiple users per workspace in later releases, although v1.0 permits only one active member.	Stored fact
Repository	Connected repository metadata including source platform, external repository identifier, name, URL, visibility, connection status, and owning workspace.	Stored fact
Branch	Repository branches, including branch name, current head commit SHA, and whether the branch is the default branch	Stored fact
AnalysisAttempt	Every attempted repository analysis, including branch, commit SHA, analysis-engine version, trigger type, execution status, timestamps, retry count, and failure information.	Stored fact
Snapshot	Immutable finalized result of a successfully completed analysis, including the associated analysis attempt, commit SHA, scan time, and finding count.	Stored fact
SourceFile, StaticMetric, ProcessMetric	Metadata describing analyzed files and the static and history-derived measurements extracted for the analyzed revision.	Stored fact

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Entity	What it holds	Stored fact or derived?
Finding	Detected technical-debt evidence including source, category, severity, location, reason/evidence, measured value, threshold, confidence where applicable, and fingerprint.	Stored fact
SATDPrediction, BugRiskPrediciton	Outputs of the machine-learning models, including SATD prediction information and the per-file bug-risk score together with the model version that produced them.	Stored fact
ScoringProfile	Workspace-configurable category weights and rule-versus-model trust setting used when deriving priorities and health scores.	Stored configuration
RuleDefinition SatdMarkerPattern, DebtCategory	Rule, category, severity, threshold, marker, and message-template definitions used by the analysis pipeline.	reference data
MLModelVerison, AnalysisEngineVersion	Model and analysis-engine provenance required to reproduce and explain analysis results.	Reference data
FileTreeNode, CodeSymbol, SourceLocation	Structural metadata required to reconstruct the analyzed repository tree and locate findings within files and code symbols.	Stored fact

Scoring is performed in the application layer instead of calculating or storing as persistent database values. The database only stores analysis facts and scoring configuration and the profile-dependent outputs are derived using a deterministic Python function.

This keeps the scoring logic independently testable and allows thresholds and weights to change without requiring database schema or SQL-view changes. If read time scoring later becomes a measured performance bottleneck, PostgreSQL may be used to pre-aggregate the required inputs and the final profile dependent calculation remain in Python. However, this optimization is not a part of the v1.0 design.

Table 9-2. Columns and tables with a rule attached.

Column or table	Rule that applies to it	Requirement
FINDING.severity and FINDING.category	everity and category are assigned when the finding is created and stored as analysis facts. They are not changed when the scoring profile changes. Rule-engine findings use the fixed category and severity defined by their rule; SATD findings receive their category from the classifier and severity from the configured SATD marker rules.	FR-8.1, FR-9.2

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

Column or table	Rule that applies to it	Requirement
FINDING.source	Identifies the detector that produced the finding. In v1.0, findings originate from either the rule engine or SATD detection. The bug-risk model produces a per-file risk score and does not create findings.	FR-8.2, FR-10
SCORE_PROFILE.weight_s and trust_s	Category weights and the rule-versus-model trust value are configurable inputs to scoring. Changing them recalculates profile-dependent outputs from existing stored facts and must not trigger a new repository scan or modify a snapshot.	FR-20, FR-21, SP-7
WORKSPACE.active_profile_id	Identifies the scoring profile currently used when deriving priorities, health scores, grades, trends, and category breakdowns for the workspace. Changing the active profile changes derived values only; stored analysis facts remain unchanged.	FR-20, FR-21
SCAN (append-only)	Finalized analysis results are immutable and append-only. A successful analysis creates a new snapshot rather than updating an earlier one. Trend, history, and delta values are therefore obtained by querying existing snapshots in chronological order. Failed or cancelled attempts may retain their execution records but must not create finalized snapshots.	FR-19, FR-21, REL-05, DBR-22, DBR-23

10. Size and Performance

Code Sage AI v1.0 is dimensioned to support at least 50 single-user workspaces, representing a baseline capacity of 50 registered users, while maintaining the specified response-time targets under normal operating conditions. The system shall support at least three repository-analysis requests concurrently. When the number of analysis requests exceeds the available processing capacity, additional requests are placed in the task queue without blocking interactive system functions such as authentication, navigation, access to analysis history, and access to previously completed results.

The architecture addresses these size and performance requirements in the following ways:

Non-blocking repository analysis:

Repository scans are executed by asynchronous worker processes rather than within the backend API request-response cycle. The API validates the request and places the analysis job in the Redis task queue, allowing the user interface and other API operations to remain responsive while repository retrieval, metric extraction, technical-debt detection, and machine-learning inference are performed.

Efficient dashboard retrieval:

A dashboard request is handled using one database query followed by an in-memory scoring pass. The database stores findings and other facts produced by a scan, while profile-dependent values such as finding priority, file debt, health score, grade, delta, and category breakdown are calculated when the results are retrieved. Therefore, loading the

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

dashboard or changing the active scoring profile does not require the repository to be scanned again.

Horizontal worker scaling:

The analysis-worker layer is the primary scaling point because repository scans consume considerably more processing, memory, and temporary storage than ordinary API requests. Additional worker containers can be deployed to increase concurrent analysis capacity without changing the frontend, API contract, or database schema. The SRS requires each component to be independently buildable and scalable.

Reduced dependence on GitHub REST quotas:

The repository-analysis pipeline retrieves the working tree and commit history through a read-only Git clone, which does not consume GitHub REST API quota. GitHub REST API calls are used only for repository metadata and use conditional requests to reduce rate-limit consumption.

Each worker requires at least 2 GB of temporary local disk space per concurrent scan for the repository clone and intermediate analysis files. This temporary storage is released when the analysis completes, fails, or is canceled.

Table 10-1. Dimensioning characteristics and performance targets.

Characteristic	Target	Source requirement
Registered users at baseline	Support at least 50 single-user workspaces, representing a baseline capacity of 50 registered users, while maintaining the specified response-time requirements.	PERF-06
Concurrent repository analysis	Support at least three repository-analysis requests concurrently. Requests exceeding the available capacity shall be queued without blocking interactive functionality.	PERF-07
Interaction feedback	Provide visible feedback within 0.1 seconds after receiving a direct user interaction.	PERF-01
Non-analysis interactions	Complete user interactions that do not involve repository retrieval or analysis within 1 second.	PERF-02
Scan enqueue	Place an initiated repository-analysis job in the task queue within 1 second.	PERF-03
Progress reporting	Provide progress and status information for an operation expected to take more than 10 seconds until it completes, fails, or is canceled.	PERF-04
Scan-progress polling	Poll the scan-status endpoint once every second while a scan is active. WebSockets and Server-Sent Events are not used in v1.0.	Communications Interface
Worker disk	Provide at least 2 GB of temporary local disk space for each concurrent repository scan and release it when the scan ends.	Hardware Interface

The frontend obtains scan progress through asynchronous HTTP polling of the scan-status endpoint at a one-second interval. Polling stops when the scan reaches a completed, failed, or canceled state. This approach provides continuous progress information without adding the deployment complexity of WebSockets or Server-Sent Events in v1.0.

The deployment shall also monitor processor usage, memory usage, persistent storage, database connections, and analysis-worker queue utilization so that resource exhaustion and performance degradation can be detected. Under

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

high analysis demand, queued scans may wait for worker capacity, but interactive parts of the application shall remain available.

11. Performance Measurements

The following values depend on the implemented system and deployment environment. They shall be measured during performance testing before the final submission rather than estimated during architectural design. The SAD template specifically requires these four measurements.

Measurement	Measured Value	Conditions to Record
Scan time per KLOC	[To be measured]	Repository language and size, commit-history size, worker CPU and memory, and model-loading state
Maximum concurrent scans per worker	[To be measured]	Worker CPU, memory, temporary disk, repository sizes, and acceptable performance degradation
P95 dashboard-read latency	[To be measured]	Number of files, findings and snapshots, concurrent-user load, and database configuration
Database size per snapshot	[To be measured]	Repository size, analyzed file count, finding count, stored evidence, and database indexes

12. Quality

This section explains how the software architecture contributes to all capabilities (other than functionality) of the system. The architecture designed to maintain a better reliability, security, maintainability, scalability and AI model quality.

12.1 Reliability

ID	Requirement	Architectural Mechanism
REL-01	Maintain high system availability	Fast API services and Celery workers are deployed separately using Docker containers, allowing individual services to restart independently.
REL-02	Prevent one repository analysis failure from affecting others	Each repository scan runs as an independent Celery task managed through Redis, isolating failures between analyses.
REL-03	Preserve analysis history	PostgreSQL stores repository versions, analysis timestamps, statuses, and generated results for future reference.
REL-04	Handle temporary failures	Celery automatically retries failed tasks caused by temporary issues before reporting a final failure.
REL-05	Prevent incomplete results from replacing valid results	Analysis results are stored only after the complete analysis pipeline is successfully completed.
REL-06	Protect existing data during external service failures	Failures related to GitHub authentication or repository access stop the current scan without affecting previous successful analyses.
REL-10	Ensure consistent analysis results	Static analysis tools such as CK, Tree-sitter, and PyDriller generate consistent results for the same repository version.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

12.2 Performance

ID	Requirement	Architectural Mechanism
PERF-01	Provide immediate feedback	The frontend provides loading indicators and status updates after user actions.
PERF-02	Provide fast dashboard access	Dashboard information is derived from the findings stored in PostgreSQL instead of running analysis repeatedly.
PERF-03	Quickly start repository scans	Scan requests are added to Redis queues immediately and processed asynchronously by workers.
PERF-04	Display analysis progress	Users can monitor the current analysis state such as Queued, Running, Completed, or Failed.
PERF-05	Maintain responsive user interaction	Heavy processing tasks are executed by Celery workers without blocking FastAPI requests.
PERF-07	Support multiple analyses simultaneously	Multiple worker instances can process different repository analyses concurrently.
PERF-09	Continue service operation during external failures	Previously generated reports remained available even when external services such as GitHub are temporarily unavailable.

12.3 Security

ID	Requirement	Architectural Mechanism
SEC-01	Secure user authentication	Users authenticate through GitHub OAuth before accessing system features.
SEC-04	Control repository access	v1.0 connects public repositories by pasting a URL and no GitHub App installation is used (SRS FR-3). Private-repository authorization is v2 (SRS SEC-04).
SEC-05	Apply minimum required permissions	The system uses read-only Git and REST access and never modifies repository resources (SRS SEC-05)
SEC-07	Protect communication	HTTPS/TLS encryption is used between clients and backend services.
SEC-08	Secure sensitive credentials	Tokens, API keys, and database credentials are stored securely using environment variables.
SEC-03 SEC-13	Ensure user data isolation	Repository data and analysis results are accessible only to authorized users.
SEC-11	Validate incoming requests	FastAPI validates API inputs before processing requests.
SEC-16	Prevent information leakage	Error messages are designed to avoid exposing sensitive system details.

12.4 Privacy

ID	Requirement	Architectural Mechanism
PRI-01	Protect repository confidentiality	Repository details and analysis results are available only to authenticated users.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

PRI-02	Secure credential management	GitHub tokens and API credentials are stored only within backend services.
PRI-03	Minimize collected data	The system processes only the information required for technical debt analysis.
PRI-04	Protect data transmission	HTTPS/TLS ensures secure communication between users and system services.

12.5 Usability

ID	Requirement	Architectural Mechanism
U-01	Provide a simple workflow	Users can connect repositories, start analysis, and view results through a clear process.
U-02	Improve result understanding	Findings are presented using severity levels, health scores, and explanations.
U-03	Support visual analysis	Charts and repository views help users identify technical debt areas.
U-04	Maintain interface consistency	Reusable frontend components provide a consistent user experience.
U-05	Show analysis progress	Users can view the current progress of long-running operations.
U-06	Support recovery from failures	Previous successful reports remain accessible even if a new scan fails.

12.6 Maintainability

ID	Requirement	Architectural Mechanism
MAINT-01	Support modular development	System components are separated into independent modules.
MAINT-02	Reduce system coupling	Presentation, business logic, and data management layers are separated.
MAINT-03	Allow future extensions	New analysis tools, programming languages, and AI models can be added with minimal changes.
MAINT-04	Support configurable scoring	Technical debt prioritization rules can be modified through configuration instead of code changes.
MAINT-05	Allow independent AI model updates	Machine learning models are maintained separately as versioned artifacts.

12.7 Supportability

ID	Requirement	Architectural Mechanism
SUP-01	Support troubleshooting	System events, scan status, and execution details are recorded through centralized logging.
SUP-02	Maintain communication consistency	Shared API models ensure consistent data exchange between frontend and backend.
SUP-03	Ensure deployment consistency	Docker containers provide the same environment across development and production.
SUP-04	Enable independent testing	Individual components can be tested separately.

Code Sage AI	Version: 1.0
Software Architecture Document	Date: 09/08/2026
CS3203-G16-SAD-v1.0	

SUP-05	Support automated deployment	The modular structure allows integration with CI/CD pipelines.
--------	------------------------------	--

12.8 Portability

ID	Requirement	Architectural Mechanism
PORT-01	Support different deployment environments	Docker containers allow deployment across different platforms.
PORT-02	Support additional languages	New parsers and analysis rules can be integrated for other programming languages.
PORT-03	Provide platform-independent access	The web-based application can be accessed through modern browsers.

12.9 Scalability

ID	Requirement	Architectural Mechanism
SCALE-01	Scale system components independently	Frontend, backend, and worker services can be scaled separately using containers.
SCALE-02	Handle heavy processing efficiently	Celery workers execute analysis tasks in the background.
SCALE-03	Manage increasing workloads	Redis queues organize and distribute analysis requests efficiently.

12.10 AI Model Quality

ID	Requirement	Architectural Mechanism
AI-01	Improve SATD detection accuracy	The SATD classifier is trained using labelled datasets and evaluated using metrics such as Precision, Recall, and F1-score before deployment.
AI-02	Provide reliable bug prediction	The bug-proneness model is trained using software metrics and validated before integration.
AI-03	Maintain reproducible predictions	The system records the model version used for each prediction.
AI-04	Support model replacement	Machine learning models are stored separately and can be updated without changing application logic.
AI-05	Separate training and prediction processes	Model training is performed offline, while the deployed application only performs inference to maintain stable performance.